

Payroll Module API Documentation

This document outlines the REST API endpoints, request payloads, and response structures for the MyHourly **Payroll Module**. It is intended for the frontend team to integrate the payroll functionality.

Models

Shared Data Types

- **PaymentMode:** Enum(`BANK_TRANSFER` , `CASH` , `CHEQUE` `UPI` , etc. - depends on backend enums)
- **PayrollStatus:** Enum(`DRAFT` , `GENERATED` , `APPROVED` , `PAID` , `CANCELLED`)
- **SalaryStructureStatus:** Enum(`ACTIVE` , `INACTIVE`)
- **EmploymentType:** Enum(`FULL_TIME` , `PART_TIME` , `CONTRACT` , `INTERN` etc.)

[!NOTE]

All currency amounts (salary, deductions, allowances) are decimal values (e.g., `30000.00`).

All dates are in `YYYY-MM-DD` format. DateTimes are ISO strings.

1. Employee Payment Details API

Manage employee bank accounts, PAN, PF, and ESI details.

Base Path: `/api/v1/payroll/payment-details`

Endpoints

Method	Endpoint	Description	Roles Allowed
POST	<code>/</code>	Create Payment Details	<code>HR_ADMIN</code> , <code>SUPER_ADMIN</code>
PUT	<code>/ {employeeId}</code>	Update Payment Details	<code>HR_ADMIN</code> , <code>SUPER_ADMIN</code>
GET	<code>/ {employeeId}</code>	Get Payment Details	<code>EMPLOYEE</code> , <code>MANAGER</code> , <code>HR_ADMIN</code> , <code>SUPER_ADMIN</code>

Method	Endpoint	Description	Roles Allowed
DELETE	/employeeId	Delete Payment Details	SUPER_ADMIN

Payloads

CreateEmployeePaymentDetailsRequest

```
{
  "employeeId": 1,
  "panNumber": "ABCDE1234F",
  "bankName": "State Bank of India",
  "accountNumber": "123456789012",
  "ifscCode": "SBIN0001234",
  "paymentMode": "BANK_TRANSFER",
  "uanNumber": "100123456789",
  "pfNumber": "KNBNG1234567000001",
  "esiNumber": "31001234567890001"
}
```

UpdateEmployeePaymentDetailsRequest

(Same as Create, but without *employeeId*)

EmployeePaymentDetailsResponse

```
{
  "id": 1,
  "employeeId": 101,
  "employeeCode": "EMP000101",
  "employeeName": "Jitendra Prajapati",
  "panNumber": "ABCDE1234F",
  "bankName": "State Bank of India",
  "accountNumber": "123456789012",
  "ifscCode": "SBIN0001234",
  "paymentMode": "BANK_TRANSFER",
  "uanNumber": "100123456789",
  "pfNumber": "KNBNG1234567000001",
  "esiNumber": "31001234567890001",
  "createdAt": "2026-08-27T10:00:00",
}
```

```
"updatedAt": "2026-08-27T10:00:00"
}
```

2. Salary Template API

Manage reusable templates for salary components (Earnings & Deductions) based on employee types.

Base Path: `/api/v1/payroll/salary-templates`

Endpoints

Method	Endpoint	Description	Query Params
POST	<code>/</code>	Create Template	-
PUT	<code>/ {id}</code>	Update Template	-
GET	<code>/ {id}</code>	Get Template by ID	-
GET	<code>/employee-type/ {employeeType}</code>	Get Template by Employee Type	-
GET	<code>/</code>	Get All Templates	? activeOnly=true/false
PATCH	<code>/ {id}/status</code>	Update Template Status	-

Payloads

`CreateSalaryTemplateRequest` / `UpdateSalaryTemplateRequest`

(Note: `employeeType` is only present in the `Create` request)

```
{
  "employeeType": "FULL_TIME",
  "basicSalary": 30000.00,
  "hra": 12000.00,
  "specialAllowance": 5000.00,
  "medicalAllowance": 1500.00,
  "travelAllowance": 2000.00,
  "bonus": 3000.00,
  "otherAllowance": 1000.00,
```

```
"pf": 1800.00,
"esi": 0.00,
"professionalTax": 200.00,
"incomeTax": 0.00,
"otherDeduction": 0.00
}
```

UpdateSalaryTemplateStatusRequest

```
{
  "active": true
}
```

SalaryTemplateResponse

(Includes all fields from the request, plus...)

```
{
  "id": 1,
  "grossSalary": 54500.00,
  "active": true,
  "createdAt": "2026-08-27T10:00:00",
  "updatedAt": "2026-08-27T10:00:00"
}
```

3. Salary Structure API

Assign templates to specific employees, handle salary revisions, and track their effective periods.

Base Path: `/api/v1/payroll/salary-structures`

Endpoints

Method	Endpoint	Description	Query Params
POST	<code>/</code>	Create Initial Structure	-
POST	<code>/revision</code>	Create Salary Revision	-
GET	<code>/ {id}</code>	Get Structure by ID	-

Method	Endpoint	Description	Query Params
GET	/employee/{employeeId}	Get Structures of Employee	?activeOnly=true/false
GET	/	Get All Structures	?activeOnly=true/false

[!TIP]

Creating a `/revision` automatically marks the prior structure `INACTIVE` and sets its `effectiveTo` to one day before the new `effectiveFrom`.

Payloads

CreateSalaryStructureRequest

```
{
  "employeeId": 1,
  "salaryTemplateId": 2,
  "effectiveFrom": "2026-08-01",
  "effectiveTo": "2027-08-01",
  "remarks": "Annual Salary Revision"
}
```

SalaryStructureResponse

(Contains all salary components (basic, hra, etc.) just like the template, plus...)

```
{
  "id": 1,
  "employeeId": 15,
  "employeeCode": "EMP00015",
  "employeeName": "Rahul Sharma",
  "salaryTemplateId": 2,
  "effectiveFrom": "2026-08-01",
  "effectiveTo": "2027-03-31",
  "grossSalary": 54500.00,
  "netSalary": 52500.00,
  "status": "ACTIVE",
  "remarks": "Annual Salary Revision",
  "createdAt": "2026-08-27T10:00:00",
  "updatedAt": "2026-08-27T10:00:00"
}
```

4. Payroll Generation & Processing API

Generate monthly payrolls, manage their states (Draft -> Approved -> Paid), and download payslips.

Base Path: `/api/v1/payroll`

Endpoints

Method	Endpoint	Description	Notes
POST	<code>/generate</code>	Generate Payroll	Can generate for all or specific employees
GET	<code>/ {id}</code>	Get Payroll by ID	-
GET	<code>/number/{payrollNumber}</code>	Get by Payroll Number	e.g. <code>PAY-2026-08-001</code>
GET	<code>/employee/{employeeId}</code>	Employee History	Gets all versions for all months
GET	<code>/month</code>	Get all by Month	<code>?payrollMonth=2026-08-01</code>
GET	<code>/status</code>	Get by Status	<code>?status=DRAFT</code>
PUT	<code>/ {id}</code>	Update a <code>DRAFT</code> payroll	Edit values before approval
PATCH	<code>/ {id}/status</code>	Update Status	E.g. Approve or Mark Paid
POST	<code>/ {id}/regenerate</code>	Regenerate Payroll	Supersedes current version and creates a new one
GET	<code>/ {id}/payslip</code>	Download Payslip	Returns <code>application/pdf</code>

Payloads

CreatePayrollRequest

```
{
  "payrollMonth": "2026-08-01",
  "employeeIds": [1, 2, 3],
  "remarks": "August 2026 Payroll",
  "saveAsDraft": false
}
```

(Leave `employeeIds` empty to generate for all active employees)

UpdateDraftPayrollRequest

Used to manually adjust days and amounts for a **DRAFT** payroll before finalization. Contains:

- `totalWorkingDays` , `workedDays` , `lopDays`
- `basicSalary` , `hra` , `specialAllowance` , `medicalAllowance` , `travelAllowance` , `bonus` , `otherAllowance`
- `pf` , `esi` , `professionalTax` , `incomeTax` , `otherDeduction`
- `remarks`

UpdatePayrollStatusRequest

```
{
  "status": "APPROVED",
  "paymentReference": "TRX-12345"
}
```

PayrollSummaryResponse (From Generate endpoint)

```
{
  "payrollMonth": "2026-08-01",
  "totalEmployees": 10,
  "generated": 9,
  "failed": 1,
  "generatedPayrolls": ["PAY-001", "PAY-002"],
  "failedEmployees": [
    { "employeeId": 3, "reason": "No active salary structure found" }
  ]
}
```

PayrollResponse

A massive object containing a snapshot of the employee, payment details, attendance, earnings, deductions, final salary, and approval info.

```
{
  "id": 1,
  "payrollNumber": "PAY-202608-001",
  "version": 1,
```

```
"active": true,  
"employeeId": 101,  
"employeeCode": "EMP000101",  
"employeeName": "Jitendra Prajapati",  
"departmentName": "Engineering",  
"designationName": "Software Engineer",  
"panNumber": "ABCDE1234F",  
"bankName": "State Bank of India",  
"accountNumber": "123456789012",  
"ifscCode": "SBIN0001234",  
"payrollMonth": "2026-08-01",  
"status": "APPROVED",  
"totalWorkingDays": 30,  
"workedDays": 28,  
"lopDays": 2,  
"payableDays": 28,  
"basicSalary": 30000.00,  
"grossSalary": 54500.00,  
"totalDeduction": 2000.00,  
"netPayable": 52500.00,  
"approvedBy": "admin",  
"approvedDate": "2026-08-25",  
"paymentDate": "2026-08-31",  
"paymentReference": "TRX-12345678",  
"remarks": "On-time processing",  
"createdAt": "2026-08-25T10:00:00",  
"updatedAt": "2026-08-25T10:00:00"  
}
```

(Simplified for brevity: contains all individual allowance and deduction fields as well).